

WEEK4-SKILL

Task1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
UW PICO 5.09 File: task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100
#define MAX_TOKEN_LENGTH 256

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

int tokenize(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0') {
        // Skip whitespace
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        // End of input
        if (input[i] == '\0') {
            break;
        }

        // Check maximum number of tokens
        if (count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            return -1;
        }

        int j = 0;

        // Read one token
        while (input[i] != '\0' && !isspace((unsigned char)input[i])) {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[count].value[j++] = input[i];
            }

            i++;
        }

        tokens[count].value[j] = '\0';
        count++;
    }

    return count;
}

void printTokens(Token tokens[], int count) {
    Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
    Exit          Justify       Where is       Next Pg      UnCut Text    To Spell
```

```
UW PIC0 5.09 File: task1.c

// Check maximum number of tokens
if (count >= MAX_TOKENS) {
    printf("Error: Too many tokens.\n");
    return -1;
}

int j = 0;

// Read one token
while (input[i] != '\0' &&
       isspace((unsigned char)input[i])) {
    if (j < MAX_TOKEN_LENGTH - 1) {
        tokens[count].value[j++] = input[i];
    }
    i++;
}

tokens[count].value[j] = '\0';
count++;
}

return count;
}

void printTokens(Token tokens[], int count) {
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n",
               i + 1,
               tokens[i].value);
    }
}

int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Check empty command
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
}

G Get Help      W WriteOut      R Read File      V Prev Pg      C Cut Text      C Cur Pos
X Exit          J Justify        W Where is      N Next Pg      U UnCut Text    T To Spell
```

```
Last login: Tue Aug 18 09:17:11 on ttys000
(eval):8: command not found: >
vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/os
vishalkarnati@VISHALS-MacBook-Air os % mkdir week4skill
vishalkarnati@VISHALS-MacBook-Air os % cd week4skill
vishalkarnati@VISHALS-MacBook-Air week4skill % touch task1.c
vishalkarnati@VISHALS-MacBook-Air week4skill % nano task1.c
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task1.c -o task1
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task1
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Parsing completed successfully.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task1
myShell> ls -l

--- Parsed Arguments ---
Argument 1: ls
Argument 2: -l

Parsing completed successfully.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task1
myShell> echo Hello World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Argument 3: World

Parsing completed successfully.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task1
myShell>
Empty command.
vishalkarnati@VISHALS-MacBook-Air week4skill %
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

UW PICU 5.09File: task2.cModified

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LENGTH 256

int parseInput(char *input,
               char args[][MAX_ARG_LENGTH]) {

    int argc = 0;
    int i = 0;

    while (input[i] != '\0') {
        // Skip whitespace
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0') {
            break;
        }

        if (argc >= MAX_ARGS) {
            printf("Error: Too many arguments.\n");
            return -1;
        }

        int j = 0;

        // Single quoted string
        if (input[i] == '\'') {
            // Skip opening quote
            i++;

            while (input[i] != '\0' &&
                   input[i] != '\') {

                if (j < MAX_ARG_LENGTH - 1) {
                    args[argc][j++] = input[i];
                }

                i++;
            }

            // Check closing quote
            if (input[i] != '\') {
                printf("Syntax Error: Unmatched single quote.\n");
                return -1;
            }

            // Skip closing quote
            i++;
        }
        else {
```

Get HelpExitWriteOutJustifyRead FileWhere isPrev PgNext PgCut TextUnCut TextCur PosTo Spell

UW PICU 5.09File: task2.cModified

```
int j = 0;

// Single quoted string
if (input[i] == '\'') {
    // Skip opening quote
    i++;

    while (input[i] != '\0' &&
           input[i] != '\') {

        if (j < MAX_ARG_LENGTH - 1) {
            args[argc][j++] = input[i];
        }

        i++;
    }

    // Check closing quote
    if (input[i] != '\') {
        printf("Syntax Error: Unmatched single quote.\n");
        return -1;
    }

    // Skip closing quote
    i++;
}
else {
    // Normal token
    while (input[i] != '\0' &&
           !isspace((unsigned char)input[i])) {

        if (input[i] == '\') {
            // Enter single quote mode
            i++;

            while (input[i] != '\0' &&
                   input[i] != '\') {

                if (j < MAX_ARG_LENGTH - 1) {
                    args[argc][j++] = input[i];
                }

                i++;
            }

            if (input[i] != '\') {
                printf("Syntax Error: Unmatched single quote.\n");
                return -1;
            }

            i++;
        }
        else {

            if (j < MAX_ARG_LENGTH - 1) {
```

Get HelpExitWriteOutJustifyRead FileWhere isPrev PgNext PgCut TextUnCut TextCur PosTo Spell

UW PICO 5.09File: task2.cModi

```
// Normal token
while (input[i] != '\0' &&
      !isspace((unsigned char)input[i])) {

    if (input[i] == '\\') {
        // Enter single quote mode
        i++;

        while (input[i] != '\0' &&
              input[i] != '\\') {
            if (j < MAX_ARG_LENGTH - 1) {
                args[argc][j++] = input[i];
            }

            i++;
        }

        if (input[i] != '\\') {
            printf("Syntax Error: Unmatched single quote.\n");
            return -1;
        }

        i++;
    } else {
        if (j < MAX_ARG_LENGTH - 1) {
            args[argc][j++] = input[i];
        }

        i++;
    }
}

args[argc][j] = '\0';
argc++;
}

return argc;
}

int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];

    printf("myShell> ");

    if (fgets(input,
              sizeof(input),
              stdin) == NULL) {
        return 1;
    }
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

UW PICO 5.09File: task2.cModified

```
    }

    i++;
}

args[argc][j] = '\0';
argc++;
}

return argc;
}

int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];

    printf("myShell> ");

    if (fgets(input,
              sizeof(input),
              stdin) == NULL) {
        return 1;
    }

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Empty command
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    int argc = parseInput(input, args);

    if (argc < 0) {
        return 1;
    }

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n",
              i + 1,
              args[i]);
    }

    printf("\nCommand parsing completed.\n");

    return 0;
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

```

vishalkarnati@VISHALS-MacBook-Air week4skill % touch task2.c
vishalkarnati@VISHALS-MacBook-Air week4skill % nano task2.c
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task2.c -o task2
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task2
myShell> echo Hello

--- Parsing Result ---
Argument 1: echo
Argument 2: Hello

Command parsing completed.
vishalkarnati@VISHALS-MacBook-Air week4skill % echo 'Hello World'
Hello World
vishalkarnati@VISHALS-MacBook-Air week4skill % echo 'Hello World
quotes'
Hello World

vishalkarnati@VISHALS-MacBook-Air week4skill %

```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

LW PICO 5.09 File: task3.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LENGTH 256

void expandVariable(char *input,
                    char *output,
                    int *j) {
    char variable[100];

    int k = 0;

    // Skip $
    input++;

    while (*input != '\0' &&
           (isalnum((unsigned char)*input) ||
            *input == '.')) {
        variable[k++] = *input;
        input++;
    }

    variable[k] = '\0';
    char *value = getenv(variable);
    if (value != NULL) {
        for (int i = 0;
             value[i] != '\0';
             i++) {
            if (*j < MAX_ARG_LENGTH - 1) {
                output[(*)++] = value[i];
            }
        }
    }
}

int parseInput(char *input,
               char args[][MAX_ARG_LENGTH]) {
    int argc = 0;

    int i = 0;

    while (input[i] != '\0') {
        // Skip whitespace
        while (isspace((unsigned char)input[i])) {
            i++;
        }
    }
}

```



```
UW PICO 5.09 File: task3.c Modified
[isspace((unsigned char)input[i])] {
    if (input[i] == '') {
        i++;
        while (input[i] != '\0' &&
            input[i] != '') {
            if (input[i] == '$') {
                expandVariable(
                    &input[i],
                    args[argc],
                    &j);
            }
            i++;
            while (input[i] != '\0' &&
                (isalnum(unsigned char)input[i]) ||
                input[i] == '_') {
                i++;
            }
        } else {
            if (j < MAX_ARG_LENGTH - 1) {
                args[argc][j++] =
                    input[i];
            }
            i++;
        }
    }
    if (input[i] != '') {
        printf(
            "Syntax Error: Unmatched double quote.\n"
        );
        return -1;
    }
    i++;
} else {
    if (j < MAX_ARG_LENGTH - 1) {
        args[argc][j++] = input[i];
    }
    i++;
}
}
args[argc][j] = '\0';
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: task3.c Modified
    }
    i++;
}
}
if (input[i] != '') {
    printf(
        "Syntax Error: Unmatched double quote.\n"
    );
    return -1;
}
i++;
} else {
    if (j < MAX_ARG_LENGTH - 1) {
        args[argc][j++] = input[i];
    }
    i++;
}
}
args[argc][j] = '\0';
argc++;
}
return argc;
}
int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];
    printf("myShell> ");
    if (!fgets(input,
        sizeof(input),
        stdin) == NULL) {
        return 1;
    }
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
    int argc = parseInput(input, args);
    if (argc < 0) {
```

UW PICO 5.09File: task3.cModified

```
    }
    return argc;
}

int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];
    printf("myShell> ");
    if (fgets(input,
              sizeof(input),
              stdin) == NULL) {
        return 1;
    }
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
    int argc = parseInput(input, args);
    if (argc < 0) {
        return 1;
    }
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n",
              i + 1,
              args[i]);
    }
    printf("\nParsing completed successfully.\n");
    return 0;
}
```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Text

Cur Pos

To Spell

```
vishalkarnati@VISHALS-MacBook-Air week4skill % touch task3.c
vishalkarnati@VISHALS-MacBook-Air week4skill % nano task3.c
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task3.c -o task3
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task3.c -o task3
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task3
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Parsing completed successfully.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task3
myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

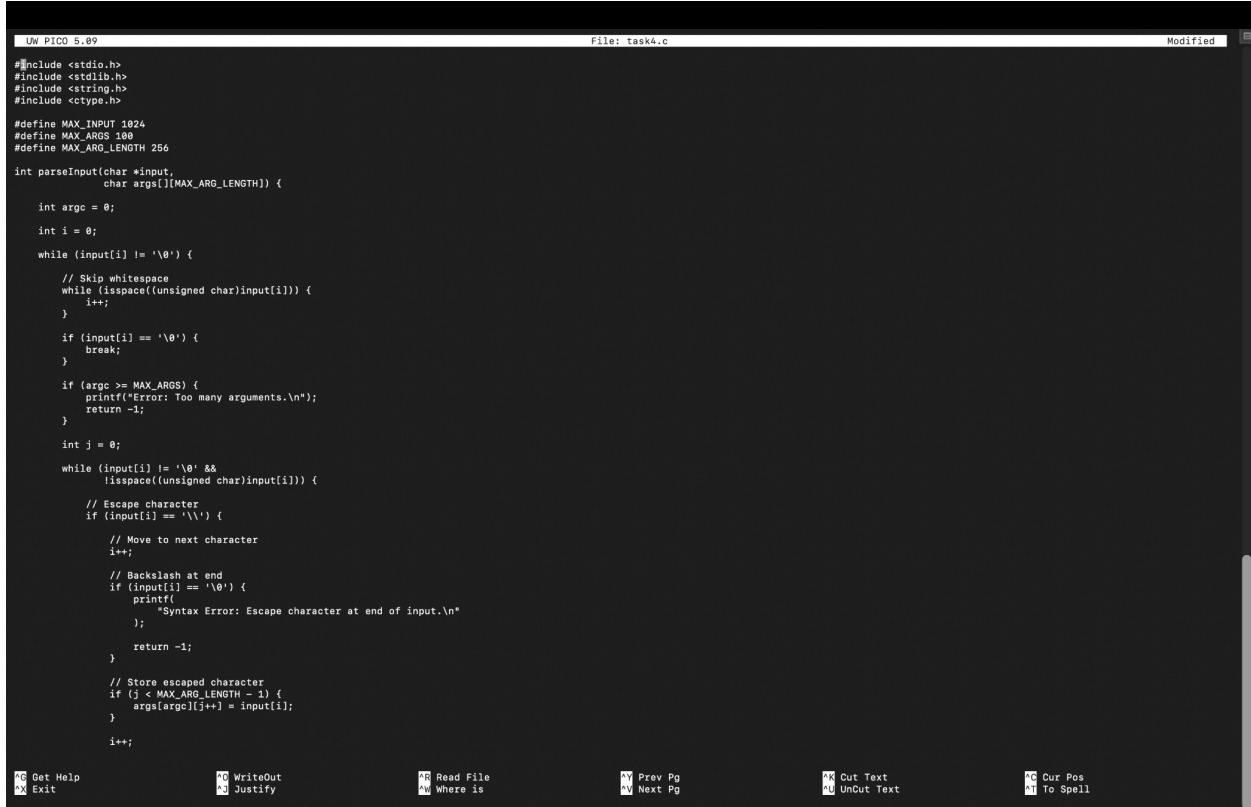
Parsing completed successfully.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task2
myShell> echo "Hello $USER"

--- Parsing Result ---
Argument 1: echo
Argument 2: "Hello
Argument 3: $USER"

Command parsing completed.
vishalkarnati@VISHALS-MacBook-Air week4skill %
```

Task 4 :

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.



```
UW PICO 5.09 File: task4.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100
#define MAX_ARG_LENGTH 256

int parseInput(char *input,
               char args[][MAX_ARG_LENGTH]) {

    int argc = 0;

    int i = 0;

    while (input[i] != '\0') {

        // Skip whitespace
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0') {
            break;
        }

        if (argc >= MAX_ARGS) {
            printf("Error: Too many arguments.\n");
            return -1;
        }

        int j = 0;

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i])) {

            // Escape character
            if (input[i] == '\\') {
                // Move to next character
                i++;

                // Backslash at end
                if (input[i] == '\0') {
                    printf(
                        "Syntax Error: Escape character at end of input.\n"
                    );
                    return -1;
                }

                // Store escaped character
                if (j < MAX_ARG_LENGTH - 1) {
                    args[argc][j++] = input[i];
                }
            }

            args[argc][j++] = input[i];
            i++;
        }

        args[argc][j] = '\0';
        argc++;
    }

    return argc;
}
```

Get Help Exit WriteOut Justify Read File Where is Prev Pg Next Pg Cut Text UnCut Text Cur Pos To Spell

```
UW PICO 5.09 File: task4.c Modified
}
int j = 0;
while (input[i] != '\0' &&
      !isspace((unsigned char)input[i])) {
    // Escape character
    if (input[i] == '\\') {
        // Move to next character
        i++;
        // Backslash at end
        if (input[i] == '\0') {
            printf(
                "Syntax Error: Escape character at end of input.\n"
            );
            return -1;
        }
        // Store escaped character
        if (j < MAX_ARG_LENGTH - 1) {
            args[argc][j++] = input[i];
        }
        i++;
    } else {
        if (j < MAX_ARG_LENGTH - 1) {
            args[argc][j++] = input[i];
        }
        i++;
    }
}
args[argc][j] = '\0';
argc++;
return argc;
}
int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];
    printf("myShell> ");
    if (fgets(input,
              sizeof(input),
              stdin) == NULL) {
        return 1;
    }
    Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
    Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: task4.c Modified
        if (j < MAX_ARG_LENGTH - 1) {
            args[argc][j++] = input[i];
        }
        i++;
    }
    args[argc][j] = '\0';
    argc++;
    return argc;
}
int main() {
    char input[MAX_INPUT];
    char args[MAX_ARGS][MAX_ARG_LENGTH];
    printf("myShell> ");
    if (fgets(input,
              sizeof(input),
              stdin) == NULL) {
        return 1;
    }
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
    int argc = parseInput(input, args);
    if (argc < 0) {
        return 1;
    }
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n",
              i + 1,
              args[i]);
    }
    printf("\nEscape sequence parsing completed.\n");
    return 0;
}
```

```
vishalkarnati@VISHALS-MacBook-Air week4skill % touch task4.c
vishalkarnati@VISHALS-MacBook-Air week4skill % nano task4.c
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task4.c -o task4
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task4
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Escape sequence parsing completed.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task4
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Escape sequence parsing completed.
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task4
myShell> echo Hello@World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello@World

Escape sequence parsing completed.
vishalkarnati@VISHALS-MacBook-Air week4skill %
```

Task 5 :

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
UW PICO 5.09 File: task5.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("myshell$ ");
        fflush(stdout);

        // Read command
        if (!fgets(input,
                    sizeof(input),
                    stdin) == NULL) {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Ignore empty command
        if (strlen(input) == 0) {
            continue;
        }

        // Exit command
        if (strcmp(input, "exit") == 0) {
            printf("Shell exiting...\n");
            break;
        }

        // Argument array
        char *args[MAX_ARGS];

        int argc = 0;

        // Split input using spaces
        char *token = strtok(input, " ");

        while (token != NULL &&
               argc < MAX_ARGS - 1) {
            args[argc] = token;
            argc++;

            token = strtok(NULL, " ");
        }

        // execvp requires NULL at the end

```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Text

Cur Pos

To Spell

```
UW PICO 5.09 File: task5.c Modified
        if (strlen(input) == 0) {
            continue;
        }

        // Exit command
        if (strcmp(input, "exit") == 0) {
            printf("Shell exiting...\n");
            break;
        }

        // Argument array
        char *args[MAX_ARGS];

        int argc = 0;

        // Split input using spaces
        char *token = strtok(input, " ");

        while (token != NULL &&
               argc < MAX_ARGS - 1) {
            args[argc] = token;
            argc++;

            token = strtok(NULL, " ");
        }

        // execvp requires NULL at the end
        args[argc] = NULL;

        // Display parent PID
        printf("Parent Process PID: %d\n",
              getpid());

        // Create child
        pid_t pid = fork();

        if (pid < 0) {
            // fork failed
            perror("fork");
            continue;
        }

        else if (pid == 0) {
            // CHILD PROCESS

            printf("Child Process PID: %d\n",
                  getpid());

            // Execute command
            execvp(args[0], args);

            // Reached only if execvp fails
            perror("Execution Error");
        }

```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Text

Cur Pos

To Spell

```
UW PICO 5.09 File: task5.c Modified

// Display parent PID
printf("Parent Process PID: %d\n",
      getpid());

// Create child
pid_t pid = fork();

if (pid < 0) {
    // fork failed
    perror("fork");
    continue;
}

else if (pid == 0) {
    // CHILD PROCESS
    printf("Child Process PID: %d\n",
          getpid());

    // Execute command
    execvp(args[0], args);

    // Reached only if execvp fails
    perror("Execution Error");
    exit(127);
}

else {
    // PARENT PROCESS
    printf("Created Child PID: %d\n",
          pid);

    int status;

    // Wait for child
    waitpid(pid,
            &status,
            0);

    if (WIFEXITED(status)) {
        printf(
            "Child exited with status: %d\n",
            WEXITSTATUS(status)
        );
    } else {
        printf(
            "Child terminated abnormally.\n"
        );
    }
}

}

}

return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: task5.c Modified

}

else {
    // PARENT PROCESS
    printf("Created Child PID: %d\n",
          pid);

    int status;

    // Wait for child
    waitpid(pid,
            &status,
            0);

    if (WIFEXITED(status)) {
        printf(
            "Child exited with status: %d\n",
            WEXITSTATUS(status)
        );
    } else {
        printf(
            "Child terminated abnormally.\n"
        );
    }
}

}

return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
vishalkarnati@VISHALS-MacBook-Air week4skill % touch task5.c
vishalkarnati@VISHALS-MacBook-Air week4skill % nano task5.c
vishalkarnati@VISHALS-MacBook-Air week4skill % clang task5.c -o task5
vishalkarnati@VISHALS-MacBook-Air week4skill % ./task5
mysHELL$ ls
Parent Process PID: 1559
Created Child PID: 1560
Child Process PID: 1560
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
Child exited with status: 0
mysHELL$ pwd
Parent Process PID: 1559
Created Child PID: 1561
Child Process PID: 1561
/Users/vishalkarnati/Desktop/os/week4skill
Child exited with status: 0
mysHELL$ mysHELL$ whoami
Parent Process PID: 1559
Created Child PID: 1562
Child Process PID: 1562
Execution Error: No such file or directory
Child exited with status: 127
mysHELL$ echo Hello
Parent Process PID: 1559
Created Child PID: 1563
Child Process PID: 1563
Hello
Child exited with status: 0
mysHELL$ abcdxyz
Parent Process PID: 1559
Created Child PID: 1565
Child Process PID: 1565
Execution Error: No such file or directory
Child exited with status: 127
mysHELL$ exit
Shell exiting...
vishalkarnati@VISHALS-MacBook-Air week4skill %
```